ITE183 - Web Systems and Technologies

LABORATORY 2

WebSocket Server

Real-Time Multi-Client Chat using the Bonezegei Scripting Language v1.3.3

Student Name:	Josh Zechariah C. Lozano
Course / Section:	ICT 3B - IT4N.1
Instructor:	BATUTAY, JOFEL A.
Date Performed:	November 1, 2026
GitHub Username:	zee7315
Repository:	https://github.com/zee7315/ITE183_Lab2_Websocket

Table of Contents

I.	Introduction and Objectives
II.	Background and Theory
III.	Environment and Tools
IV.	Procedure
V.	Source Code and Explanation
VI.	Results and Screenshots
VII.	Analysis and Discussion
VIII.	Problems Encountered and Solutions
IX.	Deliverables Checklist
X.	Conclusion

I. Introduction and Objectives

This laboratory activity focuses on building a real-time communication system using the **WebSocket** protocol. Unlike ordinary HTTP, where the client must send a request for every response it receives, a WebSocket keeps a single connection open so that the client and the server can send data to each other at any time. This activity implements such a server using the **Bonezegei Scripting Language (BSL) v1.3.3** and pairs it with a browser-based chat client. The finished system serves web pages over HTTP, upgrades connections to WebSocket, and broadcasts every chat message to all connected users.

Upon completing this laboratory, the following objectives were to be met:

- Construct and run an event-driven WebSocket server using the Bonezegei Scripting Language for full-duplex communication.
- Implement connection lifecycle handling, including socket handshakes, client identification, and disconnection tracking.
- Develop real-time message broadcasting across active connections to enable multi-client interactivity and dynamic data exchange.
- Manage the project in a public GitHub repository with the proper license, README, and language configuration.

II. Background and Theory

HTTP and the WebSocket Upgrade

Every WebSocket connection begins as a normal HTTP request. The browser sends a request that asks the server to switch protocols (an *Upgrade* handshake). If the server agrees, the same underlying TCP connection stops speaking HTTP and starts speaking the WebSocket protocol. From that moment the connection is **full-duplex**: either side may send messages whenever it wants, without waiting for a request. This makes WebSockets ideal for chat applications, live dashboards, multiplayer games, and notifications.

Event-Driven Server Design

The Bonezegei server in this lab is **event-driven**. Instead of writing a loop that constantly checks for activity, the programmer assigns handler functions to events, and the server calls them when the events occur. The handlers used in this lab are:

Event handler	Triggered when	Purpose in this lab
server.get(path, data)	A browser makes an HTTP GET request	Serve index.html for '/' and 404.html for every other path
server.onConnect(clientId)	A WebSocket client finishes the handshake	Log the new client and send a welcome message to it only
server.onData(message, clientId)	A client sends a message	Log the message and broadcast it to every connected client
server.onDisconnect(clientId)	A client closes its connection	Log which client left

Targeted Send vs. Broadcast

Two sending methods are important. `server.send(clientId, text)` delivers a message to a single, specific client, which is how the welcome message is sent. `server.sendAll(text)` delivers a message to every connected client, which is how chat messages are broadcast. Because the sender is also connected, the sender receives its own message back from the server, a behavior that is visible in the results of this lab.

III. Environment and Tools

Item	Details
Operating system	Windows (Platform reported by Bonezegei: Windows)
Language / interpreter	Bonezegei Script Interpreter v1.3.3 (Build Version v1.3.3, New Repo), by Jofel A. Batutay
Package manager	bzg (commands: `bzg install`, `bzg run`)
Dependency	`http` package (installed for arch=x86)
Terminals	Windows Command Prompt and Windows PowerShell
Browser	Google Chrome (multiple windows for concurrency testing)
Version control / hosting	GitHub (public repository, MIT License)
Server address	http://localhost:8000 (HTTP) and ws://localhost:8000 (WebSocket)

IV. Procedure

Step 1: Verify the Bonezegei installation

Bonezegei v1.3.3 was installed from the Microsoft Store. To confirm that the required version was active, `bonezegei -v` was run in a Command Prompt window. The output is shown in Figure 2.

Step 2: Create and configure the GitHub repository (Task 1)

- A new **public** repository named `ITE183_Lab2_Websocket` was created on GitHub.
- The repository was initialized with a `README.md` and the **MIT License**.
- A `.gitattributes` file containing `*.bzg linguist-language=JavaScript` was added so GitHub highlights BSL files as JavaScript.

Step 3: Create the project files (Task 2)

The project files were uploaded to the repository: `package.json`, `app.bzg`, `.gitattributes`, and the `public/` directory containing `index.html` and `404.html`. The resulting repository is shown in Figure 1. During this step the `package.json` template was corrected: the sample contained a trailing comma after the `http` dependency, which is invalid JSON, so the comma was removed.

Step 4: Install dependencies and start the server (Task 3)

The repository was downloaded to the local machine. From inside the project folder, `bzg install` was run to fetch the `http` dependency, followed by `bzg run`, which executes the `start` script (`bonezegei app.bzg`) defined in `package.json`.

Step 5: Test the system

- Three browser windows were opened at `http://localhost:8000`; each one connected and received a welcome message.
- A message ("hi") was sent from one window and observed in all windows.
- A non-existent route, `http://localhost:8000/test`, was opened to verify the custom 404 page.
- One browser window was closed to verify that the server logs disconnections.

V. Source Code and Explanation

Project Structure

```
ITE183_Lab2_Websocket/
|-- .gitattributes
|-- LICENSE
|-- README.md
|-- app.bzg
|-- package.json
`-- public/
    |-- index.html
    `-- 404.html
```

package.json

This file describes the project to the Bonezegei package manager. The `main` field names the entry point, the `scripts.start` entry is what `bzg run` executes, and `dependencies` lists the `http` package that provides the server library.

```
{
  "name": "ite183-lab2-websocket",
  "version": "1.0.0",
  "description": "Bonezegei WebSocket Server",
  "main": "app.bzg",
  "scripts": { "start": "bonezegei app.bzg" },
  "dependencies": { "http": "^1.0.0" },
  "license": "MIT"
}
```

app.bzg

```
include("lib/http.bzg");

var server = http(8000);

// HTTP EVENT HANDLERS
server.get = function (path, data) {
    print("get loc:" + path);

    if (path == "/") {
        server.sendFile("public/index.html");
    }
    else {
        server.sendFile("public/404.html");
    }
};

// WEBSOCKET (WS) EVENT HANDLERS
server.onConnect = function (clientId) {
    print("WebSocket connected! Client ID: " + clientId);
    server.send(clientId, "Welcome to WebSocket server!");
};

server.onData = function (message, clientId) {
    print("Received from Client " + clientId + ": " + message);
    server.sendAll("User " + clientId + " says: " + message);
};

server.onDisconnect = function (clientId) {
    print("WebSocket disconnected! Client ID: " + clientId);
};

server.run();
```

How it works. The server first includes the HTTP library and creates a server on port 8000. The `server.get` handler implements simple routing: the root path `/` returns the chat page, and any other path (including the browser's automatic `/favicon.ico` request) returns the custom 404 page. The WebSocket handlers manage the connection lifecycle: `onConnect` identifies the new client by its ID and greets it privately, `onData` receives a message, prefixes it with the sender's ID, and broadcasts it to all clients with `sendAll`, and `onDisconnect` records that a client has left. Finally, `server.run()` starts the event loop.

public/index.html (client)

The chat page creates a connection with `new WebSocket("ws://localhost:8000")` and reacts to four events: `onopen` (enables the input box and shows a green "Connected" status), `onmessage` (appends each incoming server message to the log), `onclose` (shows a red "Disconnected" status and disables input), and `onerror` (logs an error). Pressing Enter or clicking **Send** calls `socket.send()` after verifying the connection is open. Messages are displayed with `textContent` rather than `innerHTML`, which prevents other users from injecting HTML or scripts into the chat.

public/404.html and .gitattributes

`404.html` is a simple static page with the heading "404 - Page Not Found" shown for unknown routes. `.gitattributes` contains `*.bzg linguist-language=JavaScript`, telling GitHub's language detector to treat `.bzg` files as JavaScript for syntax highlighting.

VI. Results and Screenshots

6.1 GitHub Repository

Figure 1 shows the repository `ITE183_Lab2_Websocket` under the account **zee7315**. It is marked **Public**, and the root contains the `public` folder, `.gitattributes`, `LICENSE`, `README.md`, `app.bzg`, and `package.json`. The README and LICENSE came from the initial commit, while the other files were added through "Add files via upload".

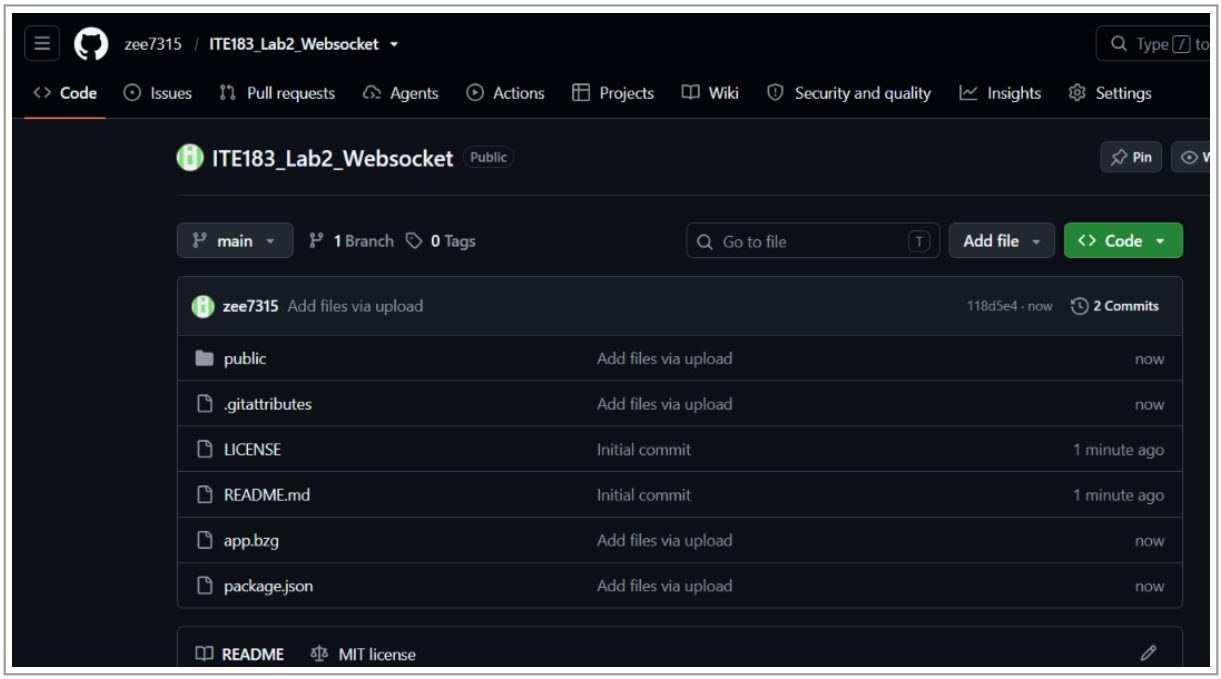


Figure 1. *Public GitHub repository containing all required project files.*

6.2 Version Verification

Figure 2 shows the output of `bonezegei -v`: *Bonezegei Script Interpreter v1.3.3*, authored by Jofel A. Batutay, Build Version v1.3.3 (New Repo), Platform: Windows. This confirms that the required version of the language is installed.

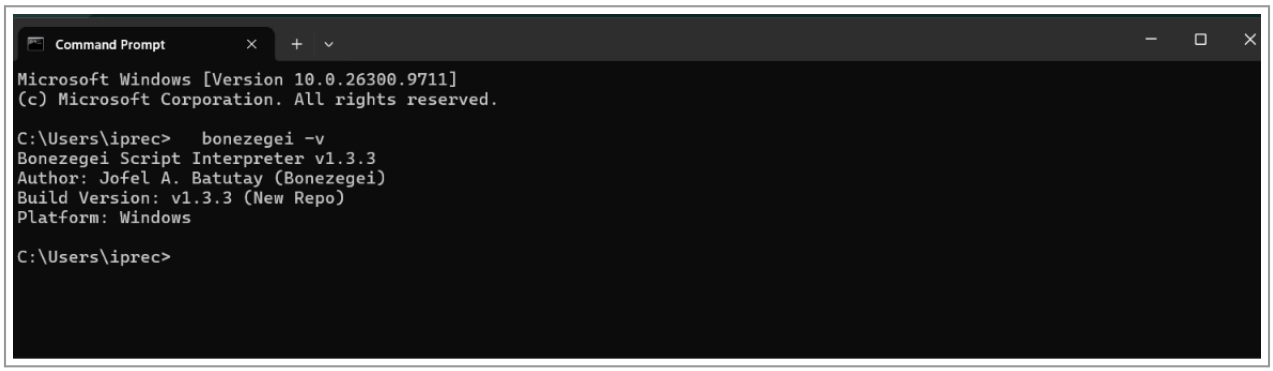


Figure 2. *Output of* `bonezegei -v` *confirming Bonezegei v1.3.3.*

6.3 Dependency Installation and Server Startup

Figure 3 shows `bzg install` reading `package.json` and installing the `http` package for arch=x86. `bzg run` then reports "Running 1 script(s) concurrently" and starts the `start` script (`bonezegei app.bzg`). The terminal stays open with no further prompt because the server is running and waiting for connections.

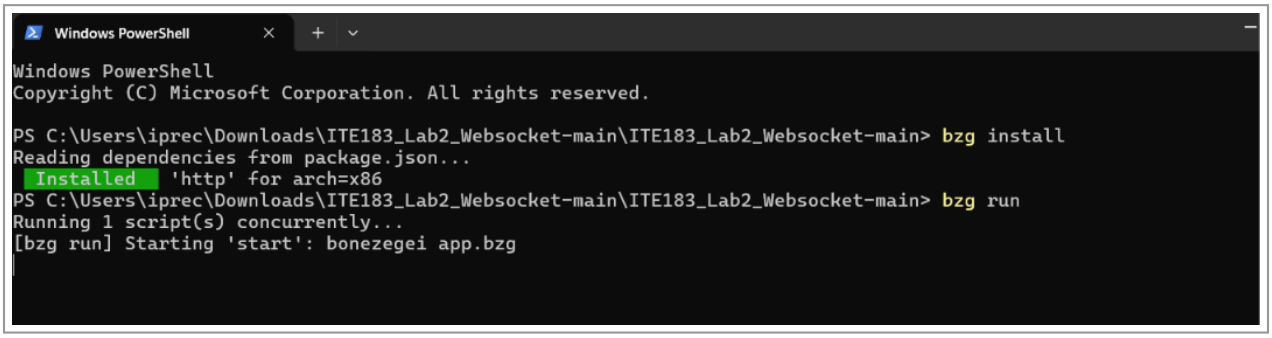


Figure 3. `bzg install` *and* `bzg run` *starting the server.*

6.4 Client Connection and Welcome Message

Figure 4 shows the chat client at `localhost:8000` immediately after loading. The status bar turned green and reads "Connected to ws://localhost:8000", meaning the WebSocket handshake succeeded. The log shows the system message "Handshake complete. WebSocket ready." followed by the server's private welcome message, "Welcome to WebSocket server!", which was sent by the `onConnect` handler.

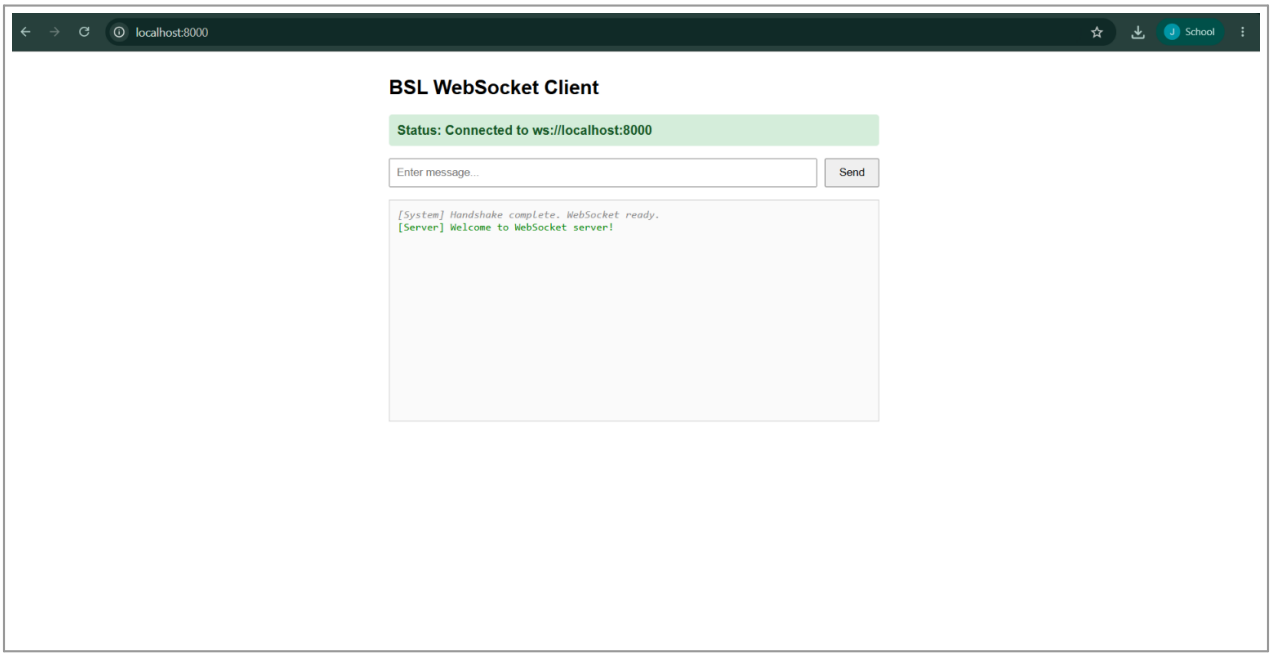


Figure 4. *Chat client connected to the server, showing the handshake and welcome message.*

6.5 Multi-Client Real-Time Broadcast

Three browser windows were connected at the same time. In Figure 5 the sending window (Client 0) typed "hi". Its log shows the locally recorded `[Client] hi` entry and then `[Server] User 0 says: hi`, which is the broadcast returned by the server. Figures 6 and 7 show the other two windows. Neither of them typed anything, yet both display `[Server] User 0 says: hi` immediately, which demonstrates that `server.sendAll()` delivers a single message to every connected client in real time. These windows also lack a `[Client]` line, confirming that they were receivers only.

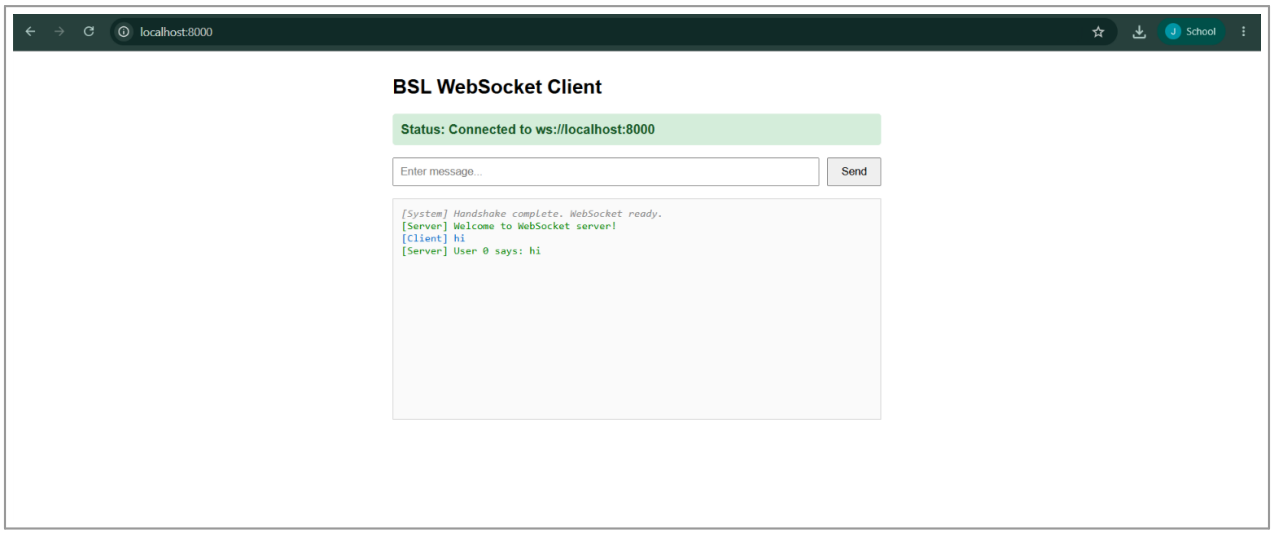


Figure 5. *Window 1 (sender, Client 0): own message and the server broadcast.*

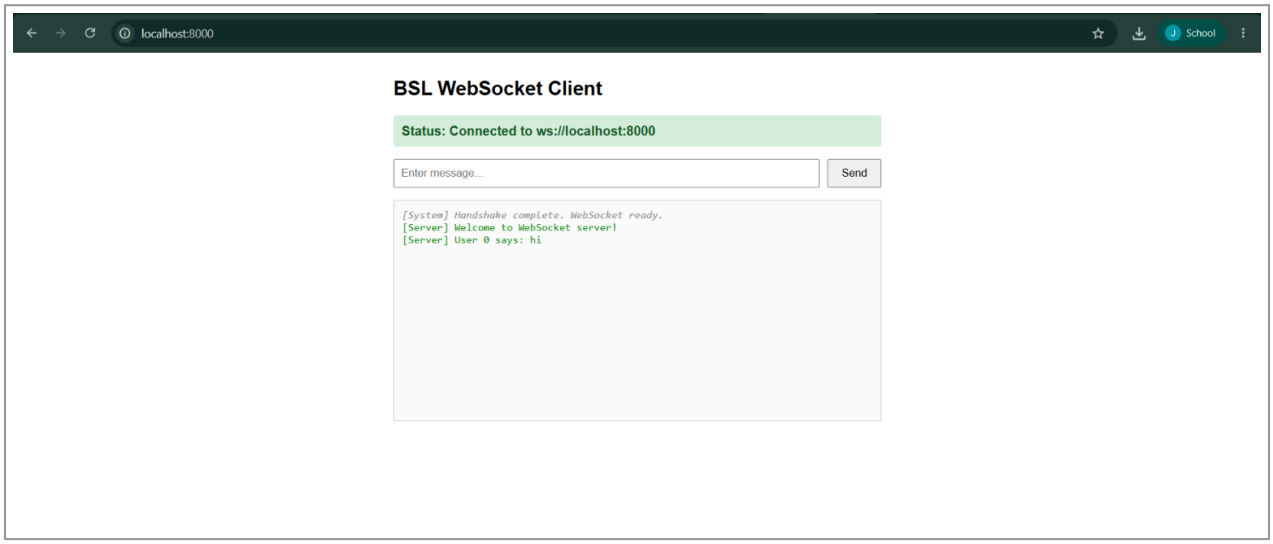


Figure 6. *Window 2 (receiver): the broadcast "User 0 says: hi" arrives without the user typing anything.*

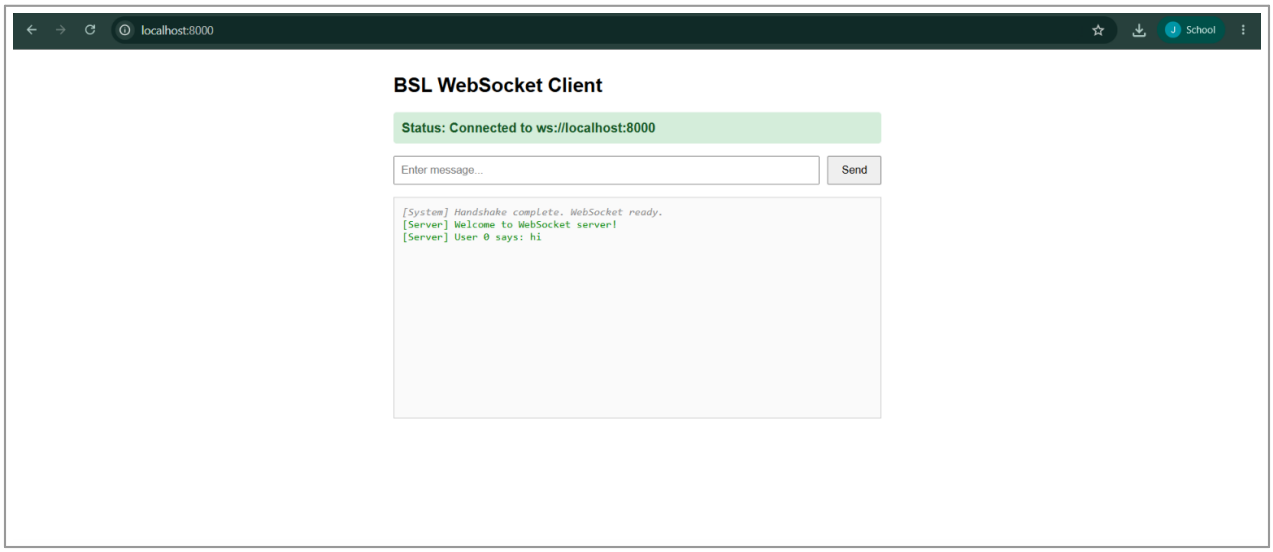


Figure 7. *Window 3 (receiver): the same broadcast message appears here as well.*

6.6 Custom 404 Page

Figure 8 shows the result of visiting `http://localhost:8000/test`, a route that does not exist. Because the path is not `/`, the `else` branch of `server.get` served `public/404.html`, displaying "404 - Page Not Found" and the explanatory message.



Figure 8. *Custom 404 page served for the undefined route /test.*

6.7 Server Log: Connection Lifecycle

Figure 9 is the server terminal after the tests. It records the full lifecycle of the session, summarized below.

Log entry	What it shows
get loc:/	A browser requested the chat page; index.html was served (occurs once per window opened).
WebSocket connected! Client ID: 0, 1, 2	Three clients completed the handshake and were assigned unique IDs.

Log entry	What it shows
get loc:/favicon.ico	Chrome automatically requests a tab icon; the server has no such route, so it returns the 404 page.
Received from Client 0: hi	The onData handler received the message sent from window 1, then broadcast it.
get loc:/test	The request for the undefined route that produced the 404 page.
WebSocket disconnected! Client ID: 2	A window was closed, and the onDisconnect handler recorded that Client 2 left.

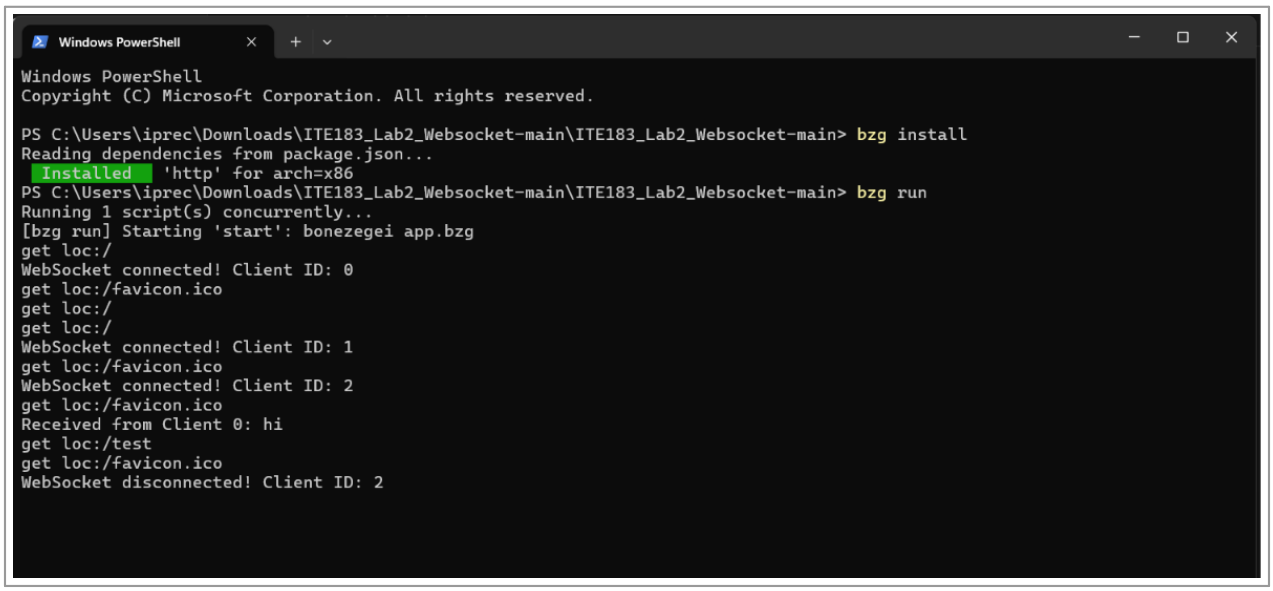


Figure 9. *Server terminal log of connections, a received message, the 404 request, and a disconnection.*

VII. Analysis and Discussion

Full-duplex communication. After the handshake, the browser never needed to ask for new messages. The server pushed the welcome message the moment the connection opened and pushed the broadcast to the receiving windows without any request from them. This is the main difference between WebSockets and plain HTTP polling.

Client identification. The server assigns each connection an incrementing ID (0, 1, 2) and passes it to every handler. This ID is used both to address a single client (`server.send`) and to label messages (`User 0 says: ...`). The server log shows that the same IDs are used consistently across connect, message, and disconnect events.

Broadcast behavior. The sender sees its message twice: once as `[Client]`, written locally by the page, and once as `[Server]`, returned by `sendAll`. This happens because `sendAll` includes the sender. If only the other users should receive the message, the server would need to skip the sender's ID. The commented-out line `server.send(clientId, "Echo: " + message)` in the source shows the alternative of replying only to the sender.

Concurrency. Three simultaneous clients were handled by a single server process without errors, and closing one window did not disturb the others. The server cleanly reported the departure of Client 2.

Routing and error handling. The routing logic is deliberately simple: exactly one path (`/`) is valid. This is why the automatic `/favicon.ico` request also returned the 404 page, as seen in the log. Browsers ignore this and the chat is not affected, but in a larger project a dedicated favicon route or a proper 404 status code would be added.

Limitations. This implementation is a learning prototype. Messages are not stored, so a window that joins late sees no history; users have no names beyond their numeric ID; the connection uses unencrypted `ws://` on localhost; and the client URL is hard-coded to `localhost`, so it only works when the browser and server run on the same machine.

VIII. Problems Encountered and Solutions

Problem	Cause	Solution
The GitHub upload showed `app.bzg` and `package.json` twice.	The files were added to the upload list more than once.	The duplicate entries were removed with the X button before committing.
Uploading the `public` folder was unclear.	The "choose your files" picker selects files, not folders; drag-and-drop is needed for folders.	The folder was dragged into the upload area; GitHub created `public/` on commit.
Invalid JSON in the provided `package.json` template.	A trailing comma followed the last dependency.	The comma was removed so the file is valid JSON.
Server-side 404 log for `/favicon.ico`.	Chrome requests a tab icon automatically.	Understood as harmless; no change required for this lab.

IX. Deliverables Checklist

Requirement	Status	Evidence
Public repository named ITE183_Lab2_Websocket	Done	Figure 1
README.md and MIT License	Done	Figure 1
.gitattributes with *.bzg linguist-language=JavaScript	Done	Figure 1, Section V
package.json, app.bzg, public/index.html, public/404.html	Done	Figure 1, Section V
bonezegei -v showing version 1.3.3	Done	Figure 2
bzg install and bzg run output	Done	Figure 3
Multi-client real-time chat broadcast	Done	Figures 4 to 7
Custom 404 page rendering	Done	Figure 8
Connection and disconnection tracking	Done	Figure 9

GitHub repository link: https://github.com/zee7315/ITE183_Lab2_Websocket

X. Conclusion

In this laboratory, an event-driven WebSocket chat server was successfully built and tested using Bonezegei Scripting Language v1.3.3. The server serves static pages over HTTP, returns a custom 404 page for undefined routes, upgrades browser connections to WebSocket, identifies each client with a unique ID, tracks connections and disconnections, and broadcasts every message to all connected users in real time. Testing with three simultaneous browser windows confirmed that a message sent from one window appeared instantly in all of them, and the server log confirmed the full connection lifecycle.

The activity gave practical experience with asynchronous event handlers, the difference between targeted sends and broadcasts, the HTTP-to-WebSocket upgrade, and the use of GitHub for project hosting, licensing, and documentation. Possible future improvements include user nicknames, message history, excluding the sender from broadcasts, timestamps, and secure `wss://` connections.